PROGRAMMING FUNDAMENTALS

A Beginner-Friendly Guide to C++ and Problem Solving

Written by Rashid Zada

Instructor



About This Booklet

This small book follows the 15 teaching weeks in the supplied Programming Fundamentals syllabus. Each week uses simple explanations, a short C++ example, expected output, and a practice task. It is designed for first-semester learners and for quick revision before class or examinations.

How to study each week

- Read the main idea and learn the key words.
- Trace the example line by line before running it.
- Predict the output, then compare it with the output shown.
- Try the practice task yourself and test your program.

Course details

Subject	Programming Fundamentals
Credit hours	3-3
Instructor	Rashid Zada
Coverage	Weeks 1 to 15 of the provided syllabus

Contents

Week 1: Overview of Computer Programming

Week 2: Designing, Testing, and Translating an Algorithm

Week 3: Algorithms into Programs

Week 4: Programming Constructs, Data Types, Input and Output

Week 5: Selection and Decision Making

Week 6: Repetition: while and for Loops

Week 7: The do-while Loop

Week 8: break and continue Statements

Week 9: Control Structures

Week 10: Functions

Week 11: Arrays

Week 12: Pointers

Week 13: Records and Structures

Week 14: File Input and Output

Week 15: Testing and Debugging

WEEK 1

Overview of Computer Programming

Learning aim: Understand programming, why it is used, and how structured and modular programming help organize solutions.

1. What is programming?

Programming means giving a computer clear instructions, written as code, to perform a task. Computers follow instructions exactly, so the instructions must be logical and correctly written.

2. Why do we program?

We use programs to solve problems, perform repeated tasks quickly, process information, and build applications, games, and websites. Common languages include C, C++, Java, and Python.

3. Structured programming

Structured programming organizes a solution into clear steps. Its three basic control structures are sequence (steps in order), selection (choosing based on a condition), and repetition (repeating steps).

4. Modular programming

Modular programming divides a large program into smaller parts, often functions. Each part performs a particular task, such as receiving input, calculating a result, or displaying output. Smaller parts are easier to test and reuse.

5. Problem-solving path

Understand the problem, identify input and output, plan the solution, write code, run it, test it, and correct errors.

Example: a simple output program

```
#include <iostream>
using namespace std;

int main() {
    cout << "Welcome to C++";
    return 0;
}
```

Expected result: Welcome to C++

Try it yourself: Write a program that displays your name and your department.

Remember: Programming begins with logical thinking. Code is the way we express that thinking to a computer.

WEEK 2

Designing, Testing, and Translating an Algorithm

Learning aim: Plan a solution before coding, represent it clearly, and translate it into C++.

1. Analyze the problem

First decide what the problem asks for. Identify the input, the process, and the output. This prevents confusion during coding.

2. Design the algorithm

An algorithm is a finite sequence of clear steps for solving a problem. A good algorithm should be understandable and should finish after a limited number of steps.

3. Pseudocode

Pseudocode describes an algorithm in simple, code-like language. It focuses on logic and does not require the exact syntax of C++.

4. Flowchart

A flowchart draws the steps using symbols and arrows. Common symbols are an oval for Start/Stop, a parallelogram for input/output, a rectangle for processing, and a diamond for a decision.

5. Test the designed solution

Try sample inputs and check whether the result is correct. Include more than one test when possible, such as small values and values that create different outcomes.

6. Translate into C++

After checking the logic, write the statements using C++ syntax, compile, run, and debug the program.

Example: add two numbers

```
Algorithm:
1. Start
2. Input A and B
3. Sum = A + B
4. Display Sum
5. Stop

Pseudocode:
Start
  Input A, B
  Sum = A + B
  Output Sum
End
```

Expected result: If A = 4 and B = 7, the expected sum is 11.

Try it yourself: Write an algorithm and pseudocode to find the area of a rectangle.

Remember: Plan first, then code. Testing the plan can save time later.

WEEK 3

Algorithms into Programs

Learning aim: Convert a step-by-step plan into a working C++ program.

1. From logic to code

An algorithm explains what steps to follow. A program expresses those steps in a programming language so the computer can execute them.

2. Identify input, process, and output

Input is the information supplied to a program. Process is the work done with that information. Output is the result shown to the user.

3. Translate each step

Input steps commonly use cin, calculations use assignment and operators, and output steps commonly use cout.

4. Compile, run, and debug

Compilation checks whether the code follows language rules. Running executes the program. Testing compares the actual output with the expected output; debugging fixes the cause of a problem.

Example: convert an algorithm into C++

```
#include <iostream>
using namespace std;

int main() {
    int a, b, sum;
    cout << "Enter two numbers: ";
    cin >> a >> b;
    sum = a + b;
    cout << "Sum = " << sum << endl;
    return 0;
}
```

Expected result: For input 4 7, the program displays: Sum = 11

Try it yourself: Turn the rectangle-area algorithm from Week 2 into a C++ program.

Remember: Every algorithm step should have a clear equivalent in the program.

WEEK 4

Programming Constructs, Data Types, Input and Output

Learning aim: Learn the basic building blocks of C++ programs and the common data types.

1. Sequence

Sequence means statements run one after another in the order written. Most simple programs begin with a sequence of input, calculation, and output.

2. Selection and repetition

Selection chooses an action based on a condition. Repetition runs a block of statements more than once. These are covered in later weeks.

3. Data types

int stores whole numbers; float and double store decimal numbers; char stores one character; string stores text; bool stores true or false.

4. Variables

A variable is a named place for a value. Choose a meaningful name and give it a suitable type, such as int marks = 80;

5. Input and output

cin receives keyboard input. cout displays information on the screen. The insertion operator << sends output, while >> takes input.

Example: input, calculate, and display

```
#include <iostream>
using namespace std;

int main() {
    string name;
    int age;
    cout << "Enter your name and age: ";
    cin >> name >> age;
    cout << name << " is " << age << " years old.";
    return 0;
}
```

Expected result: If the input is Ali 19, the output is: Ali is 19 years old.

Try it yourself: Ask the user for two whole numbers and display their product.

Remember: Choose a data type that matches the kind of value you need to store.

WEEK 5

Selection and Decision Making

Learning aim: Make a program choose an action by checking conditions.

1. if statement

An if statement runs its body only when its condition is true.

2. if-else statement

if-else provides two paths. One runs when the condition is true, and the other runs when it is false.

3. else-if ladder

An else-if ladder checks several conditions in order. The first true condition selects the branch.

4. Nested if-else

A nested if is an if statement inside another if or else block. It is useful when a second decision depends on the first.

5. switch statement

switch selects a case based on the value of an expression. Use break to prevent execution from continuing into the next case; default handles unmatched cases.

6. Conditional operator

The ?: operator is a short form for choosing between two expressions: condition ? value1 : value2.

Example: pass or fail

```
#include <iostream>
using namespace std;

int main() {
    int marks = 60;
    if (marks >= 50)
        cout << "Passed";
    else
        cout << "Failed";
    return 0;
}
```

Expected result: Passed

Try it yourself: Write an if-else program that checks whether a number is even or odd.

Remember: Use == to compare two values. A single = assigns a value.

WEEK 6

Repetition: while and for Loops

Learning aim: Repeat instructions when a task must be performed several times.

1. What is a loop?

A loop repeats a block of code. Loops reduce repeated writing and are useful for counting, totals, and processing lists of values.

2. while loop

A while loop checks its condition before each repetition. If the condition is false at the beginning, its body may not run at all.

3. for loop

A for loop places initialization, condition, and update in one line. It is especially useful when the number of repetitions is known.

4. Avoid infinite loops

Make sure the loop condition can eventually become false. If the loop variable never changes, the loop may continue forever.

Example: print numbers 1 to 5

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 5; i++) {
        cout << i << " ";
    }
    return 0;
}
```

Expected result: 1 2 3 4 5

Try it yourself: Use a while loop to print the numbers from 1 to 10.

Remember: A loop needs a condition and a way for the program to make progress toward ending.

WEEK 7

The do-while Loop

Learning aim: Use a loop that runs at least once before checking its condition.

1. How it works

A do-while loop executes its body first and checks the condition afterward. Therefore, its body runs at least once.

2. Syntax

Write do, then the block in braces, then while (condition);. The semicolon after the while condition is required.

3. When to use it

It is useful for menus or prompts that should appear at least once, then repeat only if the user chooses to continue.

4. Compare with while

while checks before the body and may run zero times. do-while checks after the body and runs at least once.

Example: count from 1 to 3

```
#include <iostream>
using namespace std;

int main() {
    int i = 1;
    do {
        cout << i << " ";
        i++;
    } while (i <= 3);
    return 0;
}
```

Expected result: 1 2 3

Try it yourself: Write a do-while loop that displays a short menu at least once.

Remember: Do not forget the semicolon after while (condition);

WEEK 8

break and continue Statements

Learning aim: Control the flow of loops using break and continue.

1. break

break immediately exits the nearest loop or switch statement. The program continues with the statement after that loop or switch.

2. continue

continue skips the remaining statements in the current loop repetition and moves to the next repetition.

3. Choosing between them

Use break when the loop should stop completely. Use continue when the loop should keep running but skip one iteration.

4. Careful use

These statements can make control flow harder to follow if overused. Use clear conditions and meaningful variable names.

Example: skip number 3

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 5; i++) {
        if (i == 3)
            continue;
        cout << i << " ";
    }
    return 0;
}
```

Expected result: 1 2 4 5

Try it yourself: Print numbers from 1 to 10 but stop the loop when the number reaches 6.

Remember: break exits the loop; continue skips only the current repetition.

WEEK 9

Control Structures

Learning aim: Understand how sequence, selection, repetition, and jump statements guide program execution.

1. Sequence

Statements run in the order in which they appear. This is the simplest flow of control.

2. Selection

Selection chooses a path according to a condition. Common forms are if, if-else, else-if, and switch.

3. Repetition

Repetition runs statements repeatedly. Common loops are for, while, and do-while.

4. Jump statements

break exits a loop or switch, continue skips to the next loop repetition, and return ends a function and may send a value back.

5. Combining structures

Real programs often combine structures. For example, a loop can contain an if statement to decide what to do with each value.

Example: print only even numbers from 1 to 6

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 6; i++) {
        if (i % 2 == 0)
            cout << i << " ";
    }
    return 0;
}
```

Expected result: 2 4 6

Try it yourself: Write a program that checks numbers from 1 to 10 and prints only those greater than 5.

Remember: Control structures decide which statements run, how often they run, and when execution changes direction.

WEEK 10

Functions

Learning aim: Divide programs into reusable pieces that perform specific tasks.

1. What is a function?

A function is a named block of code that performs a task. It helps organize a program and avoids repeating the same code.

2. Function declaration and definition

A declaration tells the compiler the function's name, return type, and parameters. A definition contains the statements the function performs.

3. Parameters and arguments

Parameters are variables listed in a function definition. Arguments are the actual values passed when the function is called.

4. Return value

A function can send a result back using return. A void function performs a task but does not return a value.

5. Reuse and readability

Give each function one clear job, such as adding numbers or displaying a menu. Small functions are easier to test.

Example: a function that adds two numbers

```
#include <iostream>
using namespace std;

int add(int a, int b) {
    return a + b;
}

int main() {
    cout << add(3, 4);
    return 0;
}
```

Expected result: 7

Try it yourself: Create a function named square that receives an integer and returns its square.

Remember: Call a function by writing its name followed by parentheses and any required arguments.

WEEK 11

Arrays

Learning aim: Store and access multiple values of the same data type under one name.

1. What is an array?

An array is a group of values of the same data type stored together. Each element is accessed using an index.

2. Indexing

In C++, array indexes start at 0. In an array of four values, valid indexes are 0, 1, 2, and 3.

3. Declaring and initializing

For example, int marks[4] = {45, 67, 88, 92}; creates an array with four integers.

4. Using loops

Loops can process every element without writing a separate statement for each one. Always keep the index within the array's valid range.

5. Why arrays help

Arrays are useful for marks, ages, prices, and other groups of similar values.

Example: display an array value

```
#include <iostream>
using namespace std;

int main() {
    int marks[4] = {45, 67, 88, 92};
    cout << marks[2];
    return 0;
}
```

Expected result: 88

Try it yourself: Create an array of five numbers and use a loop to display them.

Remember: The third element is at index 2, because indexing starts at zero.

WEEK 12

Pointers

Learning aim: Understand memory addresses and how pointer variables refer to other variables.

1. Normal variable and pointer

A normal variable stores a value. A pointer variable stores the memory address of another variable.

2. Address-of operator (&)

The expression &x; means the address of variable x. It can be stored in a compatible pointer.

3. Dereference operator (*)

The expression *p accesses the value at the address stored in pointer p, provided the pointer is valid.

4. Why pointers matter

Pointers are used in functions, arrays, dynamic memory, and data structures. They must be handled carefully.

5. Safety

Do not dereference an uninitialized or null pointer. Make sure a pointer refers to a valid object before using *.

Example: access a value through a pointer

```
#include <iostream>
using namespace std;

int main() {
    int x = 20;
    int* p = &x;
    cout << x << endl;
    cout << *p << endl;
    return 0;
}
```

Expected result: 20
20

Try it yourself: Create an integer variable, point to it, and display its value using the pointer.

Remember: & gets an address; * accesses the value at the address.

WEEK 13

Records and Structures

Learning aim: Group related information of different data types in one record.

1. What is a record?

A record groups related fields that may have different data types. In C++, a struct is commonly used to create this kind of record.

2. Fields

A student record might contain a string name, an int roll number, and a float marks value.

3. Accessing fields

Use the dot operator, such as student.name, to access a field of a structure variable.

4. Arrays and records

An array stores multiple values of the same type. A structure groups different kinds of related values into one unit.

5. Practical uses

Structures can represent students, employees, books, and other entities with several pieces of information.

Example: a student record

```
#include <iostream>
#include <string>
using namespace std;

struct Student {
    string name;
    int roll;
    float marks;
};

int main() {
    Student s1 = {"Ali", 101, 88.5};
    cout << s1.name << " " << s1.roll << " " << s1.marks;
    return 0;
}
```

Expected result: Ali 101 88.5

Try it yourself: Create a Book structure with title, pages, and price, then display its fields.

Remember: Arrays group similar values; structures group related fields that can have different types.

WEEK 14

File Input and Output

Learning aim: Read data from files and write data to files so information can be stored permanently.

1. What is a file?

A file is a collection of data stored on a storage device. Unlike values held only while a program runs, file data can remain after the program ends.

2. Input and output

Input brings data into a program. Output sends data from a program to a screen or a file.

3. File streams

Use ofstream to write to a file and ifstream to read from a file. Include the fstream header.

4. Open, use, close

Create a file stream, write or read the required data, and close the file when finished. Check that a file opened successfully in larger programs.

5. Common uses

Files can store student records, results, account information, and other data that should be available later.

Example: write a line to a file

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream outFile("student.txt");
    outFile << "Name: Ali" << endl;
    outFile << "Marks: 85" << endl;
    outFile.close();
    cout << "Data saved";
    return 0;
}
```

Expected result: Data saved. The file student.txt contains the two lines written by the program.

Try it yourself: Write a program that saves your name and roll number to a file.

Remember: ofstream writes to a file; ifstream reads from a file.

WEEK 15

Testing and Debugging

Learning aim: Check programs with sample data and correct errors so the results are reliable.

1. Testing

Testing means running a program with selected inputs and checking whether its actual output matches the expected output.

2. Debugging

Debugging means locating, understanding, and fixing the cause of a problem found during testing.

3. Syntax errors

A syntax error breaks the language's writing rules, such as forgetting a semicolon. The program generally cannot compile until it is corrected.

4. Runtime errors

A runtime error happens while a program is executing, for example attempting an invalid operation or using a resource incorrectly.

5. Logical errors

A logical error occurs when the program runs but produces an incorrect result because the steps or formula are wrong.

6. A simple testing method

Try ordinary inputs, boundary values, and cases that follow different decision paths. Compare actual output with expected output and test again after fixing errors.

Example: find and correct a logical error

```
// Incorrect if the goal is to add:
cout << 5 * 5;   // displays 25

// Correct:
cout << 5 + 5;   // displays 10
```

Expected result: Expected output for adding 5 and 5: 10

Try it yourself: Find and correct the missing semicolon in: cout << "Hello"

Remember: Testing helps reveal problems; debugging fixes them. Always test again after making a correction.

Quick Revision Checklist

- ■ I can explain the difference between a problem, an algorithm, pseudocode, and a program.
- ■ I can identify input, process, and output.
- ■ I can declare variables and choose suitable data types.
- ■ I can use cin and cout.
- ■ I can use arithmetic, comparison, and logical operators.
- ■ I can write if, if-else, else-if, and switch statements.
- ■ I can use for, while, and do-while loops.
- ■ I understand break and continue.
- ■ I can define and call a simple function.
- ■ I can store similar values in an array.
- ■ I can explain what a pointer stores and what & and * do.
- ■ I can group different fields in a struct.
- ■ I can write to a file using ofstream.
- ■ I can test a program and identify syntax, runtime, and logical errors.

Useful C++ operator reminder

Operator	Meaning	Example
+ - * / %	Arithmetic	10 % 3 gives 1
= += -= *= /=	Assignment	x += 2 adds 2 to x
== != > < >= <=	Comparison	x >= 5
&& || !	Logical	(age >= 18) && (age < 60)
++ --	Increase/decrease by one	i++
?:	Conditional choice	x > 0 ? 1 : 0

Suggested Reading

The booklet is based primarily on the supplied course syllabus and lecture notes. The following titles are listed in the syllabus or supplied course materials for further study:

- Tony Gaddis, *Starting Out with Programming Logic and Design*.
- Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*.
- Robert Lafore, *Object-Oriented Programming in C++*.
- Paul Deitel and Harvey Deitel, *C How to Program*.
- Jeri R. Hanly and Elliot B. Koffman, *Problem Solving and Program Design in C++*.

Final advice

Do not try to memorize code without understanding it. First explain the steps in your own words, then write the code, run it, and test it with different inputs. Practice a little every day.

THINK LOGICALLY.
WRITE CLEARLY.
TEST CAREFULLY.

Programming is not only typing code. It is learning how to break a problem into small steps and turn those steps into a working solution.

PROGRAMMING FUNDAMENTALS

A simple guide for first-semester students

Written by Rashid Zada
Instructor